A Formal Semantics for Reliable Collaboration among Generative AI Agents in Software Systems

Majid Babaei[1*]

[1*]School of Computing,University of the Fraser Valley,BC,Canada.

Corresponding author(s). E-mail(s): majid.babaei@ufv.ca;

Abstract

Generative workers can propose useful software artifacts without reliably respecting the dependencies, ownership boundaries, and completion conditions of a collaborative workflow. We develop a formal coordination semantics in which workers supply untrusted primitive values (text or code attributes) while a deterministic controller owns artifact structure and publication. Ranked acyclic transformation rules establish persistent correspondences between heterogeneous views. Each generated binding carries an exact dependency signature that includes selected shared context and its generation contract. A guarded transition system separates dispatch, validation, commitment, invalidation, and reconfiguration; a publication barrier checks global consistency. We prove structural integrity, current-contract acceptance, rejection of obsolete responses, dependency-relative repair locality, and a bound on generation attempts per observed dependency episode. For finite acyclic dependency networks, a topological scheduler reaches either a publishable snapshot or explicit failure under stated response and stability assumptions. A simulation theorem relates asynchronous execution to an atomic specification, and transactional configuration changes preserve the safety invariant. Counterexamples delimit the results: incomplete footprints conceal semantic changes, local validators do not imply joint consistency, and bounded retries do not ensure successful generation. A reproducible finite-state exploration checks a small asynchronous instance and detects deliberately weakened guards. The resulting guarantees concern coordination and validator-relative correctness; they do not require a probabilistic model of a language model or establish unrestricted functional correctness of generated software.

Keywords: Runtime enforcement, State machines, Model transformation, Incremental verification, Generative systems, Formal coordination

1 Introduction

A software workflow may distribute requirements, architecture, implementation, and verification among workers with different representations. A requirement is neither an operation signature nor a test oracle. Nevertheless, edits to one can invalidate the others. When a worker produces text or code using a generative model, reliable coordination requires more than a syntactically valid response: the controller must know which source state was used, which target may be written, and whether the resulting collection is ready for publication.

The difficulty is particularly visible during asynchronous execution. Suppose an implementation request is dispatched for requirement revision 1. Revision 2 arrives while generation is running, and the old answer returns later. A parser can accept that answer even though it answers an obsolete question. A second difficulty arises when two answers pass their individual validators but disagree on an interface type. A third arises when a new reviewer is added after the existing workflow appears complete. These failures involve coordination state and can be specified without assuming that a worker reasons correctly.

We study the following question: under which explicit conditions can a controller coordinate arbitrary generative proposals while preserving structural integrity, admitting only current validated values, repairing changes locally, and terminating with an honest completion verdict? Our approach separates deterministic structural transformation from untrusted value proposal. It gives each proposal an identity, a dependency signature, and a bounded attempt episode, meaning one period with unchanged dependencies. Shared knowledge is another versioned dependency. Team changes are atomic transactions over the configuration of the controller.

The formal contribution is a single semantics connecting these mechanisms. First, we define a network of guarded binding automata and a deterministic structural normalizer. Second, we prove invariants and a refinement theorem for asynchronous responses. Third, we characterize locality and finite-episode cost without claiming deterministic generated values. Fourth, we give a sufficient acyclic scheduling discipline for terminality and a safety rule for reconfiguration. Full mathematical proofs appear with their statements. A small executable explorer supplies complementary finite-instance evidence, rather than serving as a proof of the general theorems.

Runtime enforcement, incremental computation, and model consistency each supply part of the foundation [1–4]. Our concern is their intersection with proposal rejection, failed-attempt memory, context selection, and changing generation contracts. We do not claim that state machines or dependency tracking are new. The useful distinction is between an accepted value, a currently usable value, and a globally publishable snapshot. Keeping these notions separate prevents several plausible but invalid correctness arguments.

2 Foundations and running example

2.1 Formal setting

A typed view is a finite graph with stable object identities, typed primitive attributes (strings, Booleans, numbers, or inert code text), and typed references. An object identity is retained when one of its feature values changes; a reference identifies another object rather than containing a primitive value. A metamodel specifies allowed types, reference multiplicities, and well-formedness predicates. Let X denote the authoritative source views and immutable shared-context snapshots. A primitive location is an object–feature pair. Collection membership, object existence, ordering when relevant, and absence of a selected context item are observable data; treating only existing attribute values as dependencies would miss additions and deletions.

We use labeled transition systems $(\mathcal{S}, \mathcal{S}_0, \Lambda, \longrightarrow)$, consisting of a state set, initial-state set, event labels, and transition relation. A controller state is one element $S \in \mathcal{S}$. A run is a finite or infinite sequence of labeled transitions from $\mathcal{S}_0$. An invariant is a predicate true initially and preserved by every transition. Safety means that every finite reachable prefix respects an invariant. Progress properties require separate scheduling and environment hypotheses. Unbounded signatures, object identities, and primitive values make the general system an infinite-state transition system. A finite-control diagram is therefore an automaton schema, not a claim that the unrestricted workflow is a finite-state machine.

For safety, the proposal source is modeled nondeterministically: it can return any value, malformed data, a duplicate answer, or no useful answer. This overapproximates possible language-model outputs and avoids unsupported probability assumptions. Timeout is a controller event with an explicit operational assumption in the progress theorem. Rejection never makes a bad value public. It can, however, prevent successful completion.

2.2 A heterogeneous workflow

The running example is a document-service workflow. An intake view contains requests and clauses. A design view contains endpoints. An implementation view contains code patches. A verification view contains checks. An accepted request s creates endpoint e_s; each endpoint creates patch p_s; each clause c creates check t_c. The clause "an archived document cannot be overwritten" changes to "overwriting an archived document returns status 409." Status 409 denotes an HTTP conflict response. The patch must be reconsidered even if the endpoint API signature is unchanged.

Figure 1 shows the dependencies. The patch footprint, the source data made available to its worker, contains both the API signature and the exact relevant clause text. A security context can contribute a path-handling finding to patch and check generation. A reviewer added later obtains a separate view and fresh obligations. This is an illustrative formal instance; no measurements from an existing workflow are used as experimental results here.

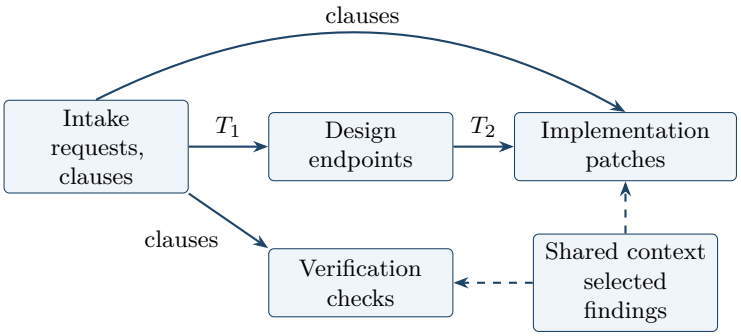


Fig. 1 A dependency network with heterogeneous views. Solid arrows carry declared source dependencies (T_1 derives endpoint interfaces; T_2 derives patches); dashed arrows carry selected shared context. The controller constructs objects and references; workers propose primitive attributes

3 Transformation contracts and dependency signatures

3.1 Deterministic structure

A configuration C contains view metamodels, rule definitions, ownership permissions, context policies, validators, generation contracts, and a publication predicate G. The base semantics requires a finite ranking of all read dependencies (A0): matches, structural expressions, observations, selectors, and validators may read only external data or strictly earlier stages. This makes availability well defined before progress is considered. A stage groups independent rules/jobs processed after their prerequisites.

A rule has a source tuple pattern, a deterministic guard, one target object, deterministic structural bindings, and zero or more generated primitive bindings. Multi-source matches range over a product of source object sets, rather than an untyped union. Rule identifiers and source identities define the match key

$$q = (r, \mathrm{id}(s_1), \ldots, \mathrm{id}(s_m)). \tag{1}$$

For each current match, a deterministic injective constructor produces target identity $\iota(q)$ and persistent trace $q \mapsto \iota(q)$. For example, $(\mathsf{RequestToEndpoint}, \mathsf{request17})$ identifies one rule match. Producer namespaces are disjoint ranges of generated identifiers. References resolve through traces only when the referenced match exists. A missing mandatory target makes normalization fail, rather than producing a dangling reference.

Let N_C be the partial structural normalizer over a coherent view snapshot. It is partial because inadmissible candidate snapshots are rejected. It computes the complete current match set for enabled stages $\mathcal{K}$, creates missing targets, removes obsolete targets and their owned traces, and computes deterministic attributes and references. It preserves accepted generated values only for surviving target bindings whose contract identity remains compatible. Its range is an internal skeleton: required generated primitive attributes may contain $\perp$, meaning not yet generated. The same symbol

later denotes unavailable observations; these are distinct typed uses of an undefined marker. External references to deleted generated objects must be removed, redirected by an explicit rule, or cause normalization failure. Deletion is not presumed harmless.

Definition 1 (Structural admissibility) Configuration C is structurally admissible on its current view snapshot when N_C terminates, is deterministic (the same snapshot gives the same structure) and idempotent (a second normalization changes nothing), has exactly one owned target and trace per match key, creates only well-typed references, and satisfies all structural multiplicity constraints. Generated bindings may write only declared primitive slots. Every write has one configured owner. A global predicate may impose further constraints beyond these structural conditions.

This definition is a proof obligation for a concrete transformation engine. We do not infer it for arbitrary transformation programs. A simple sufficient implementation allocates all targets first and resolves references second, using pure guards, disjoint producer namespaces, and total well-typed structural expressions. ATL motivates this allocation/resolution separation [5]; unrestricted imperative ATL programs are outside the definition. For dependent stages, normalization reads only usable upstream values; missing upstream values block the dependent stage. Generated values cannot alter their own stage's match set.

Theorem 1 (A constructive structural kernel) *Consider finite source graphs and finitely many single-target rules whose guards and structural expressions are pure and terminating. Suppose target identities $\iota(q)$ are injective, producer namespaces are disjoint, primitive expressions have declared types, and each reference expression returns a finite set of current match keys of the required target type and multiplicity. Enumerating matches, allocating all targets, and then resolving references constructs an admissible active skeleton. The result is unique and idempotent when generated slots are held fixed.*

Proof Each source product is finite, so terminating guard evaluation yields a finite match set. Allocation creates one target per key; injectivity and disjoint namespaces exclude collisions. The trace is exactly the graph of ι restricted to that set. Each reference key is in this domain by hypothesis, so second-phase lookup succeeds and preserves its declared type and cardinality. Typed primitive expressions preserve attribute types. Purity fixes the match set and structural values independently of enumeration order. Repeating the construction uses the same keys, identities, references and values, proving idempotence. Deleting the complement of the current key set gives exactly current coverage, with all references recomputed in the same transaction. Induction over the finite rank order proves the construction for enabled stages. □

The key-range and multiplicity conditions are checked on each candidate snapshot. Failure rejects that candidate; the theorem does not require deciding them universally for arbitrary programs.

If a prerequisite is unavailable, its dependent stage is suspended and contributes no active targets, traces, or jobs. Old accepted pairs remain only in a private cache,

unavailable to consumers. Rank-order normalization suspends all affected descendants, so active references never point into suspended stages. On reactivation, stable keys reconstruct objects; cached pairs may be restored only when observation, contract, and authority are compatible. Each reactivated job receives a new episode nonce. A set Q records required suspended stages, preventing empty job sets from being mistaken for completion.

3.2 What a binding depends on

Write $j = (q, b)$ for a generated job, consisting of a match key and identifier for one declared generated attribute binding. Let Z be the coherent readable snapshot combining authoritative X and currently usable generated predecessors, constructed from lower to higher rank under A0. Its effective observation is

$$o_j = \mathrm{obs}_{C,j}(Z) = (F_j(Z),\, \mathrm{select}_{C,j}(Z),\, \eta_{C,j}). \tag{2}$$

Here F_j is a pure total source-footprint expression, selection includes authorized context items and explicit absence markers, and $\eta_{C,j}$ identifies the complete generation/validation contract. The latter covers the instruction, schema, validator and its environment, source expression, selection policy, and generation settings whose change requires regeneration. A changed configuration need not invalidate every job, but it must change η for every affected job.

An API signature such as `put(id,body)` is an interface attribute. The dependency signature introduced next instead identifies all inputs used to generate an attribute. The signature $\sigma_j = \mathrm{enc}(o_j)$ is an injective canonical encoding: an unambiguous stable serialization in which different observations cannot share an encoding. In the mathematics it is exact equality, not a fixed-length hash. An implementation can store the encoding or use a digest with collision detection; substituting a bare cryptographic digest makes the corresponding results conditional on absence of relevant collisions. Context version numbers are audit information. They enter o_j only if the generator or validator actually observes them. Otherwise equal selected content across different context versions has equal signatures.

Footprints describe dependency functions, not just a list of locations visited on a previous execution. A query "all findings for endpoint e" depends on the selected set's membership and its contents. Either recomputing that query or maintaining a sound index over its membership dependency is necessary to discover a newly added finding. An access-denied or unavailable required context gives observation $\perp$ and blocks dispatch. Replacing missing context silently with empty text changes the contract.

Definition 2 (Footprint adequacy) For a desired Boolean domain relation $R_j(Z, v)$ on readable snapshots and proposed values, a footprint is adequate if there exists $\widehat{R}_j$ such that $R_j(Z, v) = \widehat{R}_j(o_j, v)$ for all admissible Z, v. A validator $V_j(o, v)$ is sound for that relation if $V_j(o, v) \Rightarrow \widehat{R}_j(o, v)$.

Table 1 Assumptions and the guarantees that require them

ID	Assumption	Used for
A0	Finite ranked read dependencies; suspension of unavailable stages	Defined semantics
A1	Admissible deterministic normalizer; primitive-only worker writes	Structure
A2	Exact effective signatures; all relevant contract changes included	Freshness, locality
A3	Linearizable controller transactions and immutable observations	Commit safety
A4	Total validators over recorded observations; declared stable environment	Acceptance
A5	Finite acyclic complete dependency graph, single producers, stage stability	Terminality
A6	Inputs/configuration stabilize; controller progresses fairly; requests settle	Progress
A7	Adequate footprints and sound local/global validators	Domain correctness

In plain language, two snapshots with the same observation must give the same desired-validity judgment for any fixed proposal. Adequacy is a semantic obligation, whereas signature equality is a mechanical test. In the example, a footprint containing only a signature cannot be adequate for a requirement about status 409 when two source states have equal API signatures and different status clauses. No freshness theorem repairs an inadequate specification.

3.3 Assumptions and guarantees

Table 1 collects assumptions used later. They are independently meaningful: disjoint ownership protects writes, exact signatures protect dependency comparison, and sound validation connects acceptance to a domain property. None entails the others.

4 A guarded coordination machine

4.1 State and local automata

The controller state is $S = (C, X, Y, L, B, P, A, H, Q, \nu, \rho)$: configuration, authoritative inputs, active target skeleton, traces, job/cache records, active requests, audit history, optional published snapshot, required suspended stages, fresh-nonce allocator, and semantic revision. At every transition boundary active structure is normalized. A record holds accepted value, stamp, and evidence pointer (v, a, γ) or none, failed signature f or none, episode identifier e, episode signature s, and attempts $n \in \{0, \dots, k\}$ for fixed $k \geq 1$. Evidence γ identifies immutable observation, contract, owner, and successful type/validator checks. An episode is a maximal consecutive period with one signature while the job is active; returning to an older signature or reactivating a suspended job starts a new episode. Nonces are allocated monotonically by ν, so request identity $u = (j, e, n)$ is never reused after deletion or recreation.

A request stores immutable (u, j, e, σ, o) and its designated owner. At most one request is active for a job episode. Dispatch history is retained in A after a request leaves P, allowing late and duplicate deliveries to be recognized. Cancellation prevents commitment; it need not stop remote computation. Attempts count dispatched requests even if they subsequently time out or become obsolete.

Reconciliation is a finite rank-order function. It rebuilds Q from current stage statuses, removing entries for stages that become enabled. An enabled stage with zero matches is settled successfully and is not suspended. At each stage it checks prerequisite usability, suspends unavailable stages and adds them to Q, or constructs the stage using the checked kernel. It compares each active job's new signature to its stored episode signature s. A difference, new job, or reactivation cancels its request, allocates e from ν, sets s to the new signature, clears f, and sets $n = 0$. Equal signatures of surviving active jobs preserve counters and evidence. Deletion removes active jobs and cancels requests; historical evidence remains in A. Thus no unavailable observation is sent to a worker. Semantic mutations increment ρ; currentness of H is derived by comparing its revision to ρ.

Let $\mathcal{J}(S)$ contain only active jobs; suspended/cache-only records belong to B but not $\mathcal{J}(S)$. Freshness compares the previously accepted stamp with the current dependency signature. For an active job, define

$$\mathrm{fresh}_j(S) \iff B_j \text{ has } (v, a, \gamma) \ \wedge \ a = \sigma_j(S), \tag{3}$$

$$U_j(S) \iff j \text{ active} \wedge \mathrm{fresh}_j(S) \wedge \bigwedge_{i \prec j} U_i(S), \tag{4}$$

$$O(S) = \{j \in \mathcal{J}(S) : \neg U_j(S)\}. \tag{5}$$

Here $i \prec j$ ranges over declared generated prerequisites; the empty conjunction is true. Evaluate these definitions by rank A0, so they are not cyclic equations. Inactive jobs are unusable. Current validity is deliberately absent from U: Theorem 2 derives it from acceptance evidence. Validators are total and deterministic on their declared observations; validation involving external tools must include the relevant tool versions and environment in the contract or use an immutable certificate checked against it. A cached historical value may remain for audit, but a downstream consumer may read it only through the U_j gate. If an upstream value is unusable, descendants become unusable as well, even if old bytes are still stored.

Figure 2 summarizes a job's control states. They are derived from record contents; the formal guards below determine the transitions. A dependency change also cancels a Pending request and restarts the job in Dirty. Suspension and reactivation are lifecycle operations outside the four local states. The failed state is terminal for an unchanged episode, not a successful accepting state. A worker cannot transition a job directly to fresh.

For example, a record may have accepted stamp $a = \sigma_0$ while its episode signature is $s = \sigma_1$ after an edit; it then holds a historical value and requires repair. The nonce e distinguishes this episode from an earlier occurrence of σ_1. Semantic revision ρ changes on authoritative mutations and need not equal any one source document's version.

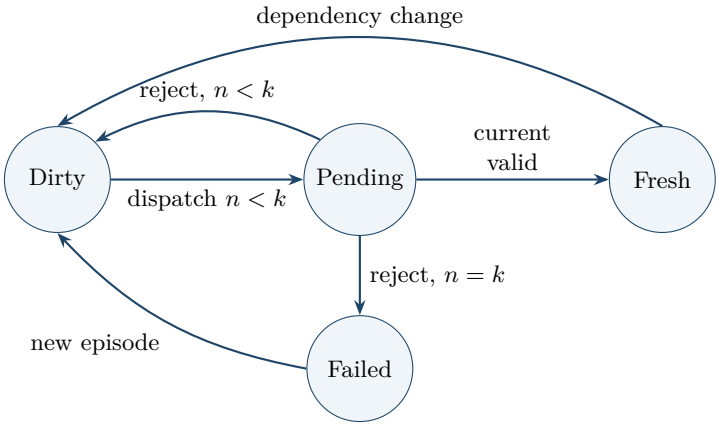


Fig. 2 Binding control automaton schema. Dependency changes reset any active state to Dirty; the Fresh reset is illustrated. Obsolete deliveries cannot write a slot. Suspension removes the active job; reactivation constructs a fresh episode

Table 2 Key notation for the controller and each generated job

Symbol	Meaning
C, X, Y, L	Configuration, authoritative input/context, active skeleton, active traces
B, P, A, H	Job/cache records, active requests, immutable history/evidence, published snapshot
Q, ν, ρ	Required suspended stages, fresh-nonce allocator, internal semantic revision
$j = (q, b)$	Match key paired with one generated attribute binding
o_j, σ_j	Current effective observation and its exact encoding
(v, a, γ)	Accepted value, accepted observation stamp, evidence pointer
(e, s, n, f)	Episode nonce, episode signature, attempts, failed signature
U_j, O	Current usability and set of active unusable jobs

4.2 Transition rules

Every transition is an atomic controller transaction. Changes to input, structure, and affected job eligibility form one transaction or use a revision barrier that prevents intermediate states from being consumed. We write $S \xrightarrow{\lambda} S'$ for the following rules; all unmentioned components are unchanged.

Edit and normalize.

An authorized edit creates X'. Compute N_C on the updated coherent snapshot, update the job set, reconcile signatures and dependent usability, cancel obsolete active requests, and advance ρ so any previous publication becomes historical. If normalization fails, keep the previous coherent state and record the rejected edit. A deletion removes owned jobs and traces and cancels their requests. A later delivery for a nonexistent or recreated job fails the request-identity check.

Dispatch.

For $j \in O(S)$, require current observation $o \neq \perp$, all required upstream jobs usable, no active request, no failed stamp equal to σ_j, and $n < k$. Increment n and add a request (u, j, e, σ_j, o) to P. The worker receives only the declared observation and static instruction. The controller records its owner and identity before sending the request.

Resolve and commit.

For a returned (u, v), require u active in P, the same existing job and episode, the configured owner still authorized, and, writing σ_u for the signature stored in request u,

$$\sigma_u = \sigma_j(S), \qquad o_j(S) \neq \perp, \qquad V_j(o_u, v). \tag{6}$$

Also require all upstream prerequisites usable and a deterministic type check for the primitive slot. Tentatively install (v, σ_u, γ) in a private candidate and run complete downstream rank-order normalization and reconciliation, because generated attributes may affect later matches or references. Commit the entire candidate only if all enabled structural checks succeed; otherwise reject it as a failed attempt. A successful commit advances ρ, clears failure and removes u. Workers supply only primitive proposals; structural changes belong to the controller.

If the request is active and current but validation, typing, or candidate normalization fails, remove it; at $n = k$ set $f = \sigma_j$, otherwise leave the job eligible. An active current timeout follows the same failure path. An inactive, unauthorized, or obsolete response is discarded; reconciliation already canceled its authority at the associated mutation. Delivery of a duplicate has no effect on semantic state.

The equality test and the slot write must share a linearization point with edits and configuration updates. Testing before acquiring a write lock is insufficient. Validation may run outside the transaction on immutable o_u, but its result can be used only after rechecking request activity and Equation (6) inside the transaction. An authentic worker response is not itself evidence that these conditions hold.

Publish.

Let $G(C, X, Y, B, L)$ be a total global predicate evaluated over an immutable candidate snapshot. It includes all mandatory joint constraints and required executable checks. The completion guard below means: no required stage is suspended, no active job is dirty, no active request is pending, and all joint checks pass. Canceled messages still in the network are historical and do not belong to P. Set

$$\begin{aligned} \text{Publishable}(S) \iff & \text{ structural normal form } \wedge\ Q = \emptyset \\ & \wedge\ O(S) = \emptyset\ \wedge\ P = \emptyset\ \wedge\ G(C, X, Y, B, L). \end{aligned} \tag{7}$$

Only a Publish transition satisfying Equation (7) installs H with that exact snapshot and revision. If G is evaluated outside the transaction, installation compares the candidate revision with current ρ and retries or rejects on mismatch. Its tool environment must be immutable and execution total, just as for local validation. Later edits leave

H as a historical snapshot but clear any claim that it represents the current state. Publication means that the specified completion contract holds for that revision.

Configuration transaction.

A candidate C' is checked for structural admissibility, ownership, policy compatibility, contract invalidations, mandatory ranked-read condition A0, and, when progress is required, the additional scheduling discipline of Section 7. Atomically install C', normalize, migrate compatible records, start new episodes for changed signatures, cancel every request whose job, authority, or contract changed, and advance ρ, even if only G changed. Reject the entire candidate if any check fails. Reusing a value under a new validator requires revalidation, not merely retaining its previous success bit.

5 Safety and asynchronous refinement

5.1 Inductive safety

We distinguish a historical acceptance invariant from current usability. A committed value need not remain fresh after an edit. The controller must prevent its use until repaired. Saying that every stored value is always fresh would be false in precisely the change scenarios of interest.

Theorem 2 (Controller safety) *Under A0–A4, from an initially normalized state with no accepted generated values, every reachable state satisfies: (i) active targets and traces have unique producers and valid structural references; (ii) each accepted record has a recorded authorized successful validation against its recorded observation and contract; (iii) every usable value satisfies the current validator for the current observation; (iv) every committed response was current and authorized at its commit linearization point; and (v) every installed published snapshot satisfied Equation* (7) *at installation.*

Proof Induct on transition length. Initially (i) is A1 and the remaining claims are vacuous. Edit/normalize preserves (i) by A1 and retains acceptance evidence only as historical records. It recomputes usability, so a changed observation cannot make an old pair usable unless its exact observation and validator contract still agree. Dispatch changes no accepted pair or structure. A successful Resolve tentatively writes the owned primitive slot and normalizes its dependent active structure before admitting the entire candidate; Theorem 1 or A1 establishes (i) for the new active graph, while a rejected candidate leaves it unchanged. Cancellation of suspended jobs preserves request authority; active request identity and authorization establish the provenance part of (ii), and the validator and exact-signature tests establish (iii) and (iv). A failed or discarded response writes no accepted pair. Timeouts change only attempt state. Publish tests the exact snapshot it installs, establishing (v). Configuration transactions normalize under the new admissible configuration, invalidate incompatible evidence, and cancel obsolete authority; they therefore preserve (i)–(iv) and do not relabel a historical publication as current. These are all transitions. In (iii), current equality of the encoded observation implies equality of validator inputs and contract by A2 and A4. □

Corollary 3 (Validator-relative domain correctness) *If a usable accepted value v has an adequate footprint and sound validator in the sense of Definition 2, then $R_j(Z, v)$ holds. If the global predicate is sound for a desired snapshot relation R_G, every published snapshot satisfies R_G.*

Proof Theorem 2(iii) supplies $V_j(o_j, v)$. Soundness yields $\widehat{R}_j(o_j, v)$; adequacy identifies it with $R_j(Z, v)$. The second statement applies global soundness to the successful publication guard. □

Example 1 (Limits of parsing and test evidence) A generated patch returning 200 for every request may parse and type-check while violating the status-409 clause. Such validators are not sound for that clause. A generated test that fails on one empty stub need not detect an incorrect implementation. Similarly, locally accepted endpoints using different document-identifier types can violate a joint interface constraint. The global predicate must check that constraint, or a certified cross-binding validator must cover it. Rejection at the publication barrier is an honest blocked outcome; it is not a proof that automatic repair will succeed.

5.2 Refinement to atomic accepted updates

Define an abstract machine $\mathcal{A}$ whose state consists of configuration, coherent input, structure, accepted records, attempt/failure records, and publication. Its *Accept* transition may choose any proposed value satisfying the current observation's validator and ownership condition and normalize the tentative update using the same checked kernel, and admit it only on success. Its *Attempt* and *Fail* transitions account for budget use and exhaustion; its Edit, Reconfigure, and Publish transitions have the guards above. It has no remote requests. This abstract specification permits many values and schedules; it does not select one canonical generated artifact.

Let $\alpha(S)$ erase requests and audit-delivery detail, retaining the semantic and budget records. Abstract publication drops the concrete requirement $P = \emptyset$, since requests are erased. It therefore permits publication at least whenever the concrete machine does. Concrete dispatch maps to an abstract budget increment, rather than a stutter; claiming that dispatch stutters while retaining an incremented counter would be incorrect.

Table 3 fixes the abstract action relation. Let s_j be the recorded episode signature and $\mathcal{R}$ the checked rank-order normalization/reconciliation function. Abstract Fail overapproximates failed attempts; it does not infer the existence of an erased request. This specification constrains admitted effects while abstracting the transport protocol.

Theorem 4 (Forward simulation) *Under A0–A4, every concrete controller run projects through α to an abstract run, allowing stuttering (an erased step leaves the abstract state unchanged). In particular, asynchronous responses cannot introduce a value forbidden by the atomic accepted-update specification.*

Table 3 Abstract action guards and effects; unmentioned fields are unchanged

Action	Guard	Effect
Attempt(j)	Active dirty job, available observation, $n_j < k$, $f_j \neq s_j$	$n_j \leftarrow n_j + 1$
Accept(j, v)	Active authorized dirty job, $0 < n_j \leq k$, current type/validator checks, candidate normalization succeeds	Install current evidence, clear failure, apply $\mathcal{R}$
Fail(j)	Active dirty job, $0 < n_j \leq k$	If $n_j = k$, set $f_j \leftarrow s_j$
Edit / Reconfigure	Authorized candidate passes checked construction	Install candidate, apply $\mathcal{R}$
Publish	Normal form, $Q = O = \emptyset$, G true	Install immutable revision snapshot

Proof Consider one concrete transition. Dispatch maps to abstract Attempt because both increment the same current-episode budget; request creation is erased. Successful Resolve maps to Accept: the concrete current-signature test implies that its immutable observation is exactly the current abstract observation, and the remaining tests imply the abstract validator and ownership guards. Both actions apply the same checked candidate normalization. Current rejection or timeout maps to Fail, including the exhausted-budget mark if necessary. Obsolete or duplicate delivery changes no retained record and maps to a stutter; signature reconciliation belongs to the associated edit/configuration transition in both machines. Edit and Reconfigure use the same normalized snapshots and migration in both machines. Concrete Publish has all abstract global, structural, and freshness guards, so maps to abstract Publish. Initial projections agree. Concatenation yields a projected run. No converse or liveness equivalence is claimed. □

If both cancellation of obsolete requests and commit-time current-observation checks are removed, dispatch on o_0, edit to o_1, and delivery of an old valid answer can commit obsolete provenance. Either defense alone suffices in the one-job finite model of Section 10; their redundancy must not be mistaken for two independent counterexamples. A check/write race outside the transaction can similarly bypass the intended guards. Without unique request identities, a duplicated reply can overwrite a newer accepted pair after deletion and recreation. These counterexamples explain the guards rather than assuming reliable delivery.

6 Local repair and bounded generation

6.1 Dependency-relative locality

For two normalized readable snapshots with the same job identities and contract, define

$$D(Z, Z') = \{j : o_j(Z) \neq o_j(Z')\}. \tag{8}$$

Add newly created jobs as dirty and remove deleted jobs; structural matching is checked separately. This is the exact direct impact set relative to the declared observation functions. It is not a claim of minimum semantic repair over all conceivable implementations.

Theorem 5 (Exact direct invalidation) *Assume all surviving jobs are usable before one authorized edit; no job is pending; and no generated values are changed during the edit transaction. For each surviving job (active in both snapshots) with available prerequisites after normalization, its accepted stamp becomes stale if and only if it belongs to $D(Z, Z')$. Within that surviving/available set, jobs outside D retain their validator-relative value. Descendants of unusable prerequisites are additionally blocked until those prerequisites are repaired.*

Proof Before the edit, $a_j = \mathrm{enc}(o_j(Z))$. After it, the stored pair is unchanged and the current signature is $\mathrm{enc}(o_j(Z'))$. Injectivity gives $a_j \neq \sigma_j(Z')$ exactly when $o_j(Z) \neq o_j(Z')$. If observations agree, validator input and contract agree, so the former successful validation still applies. The prerequisite gate explains descendant blocking independently of byte-level stamp inequality. New and deleted jobs are handled by their separate construction/removal rules. □

For a static sound dependency graph, an edit's transitive descendants form a conservative repair cone. An unchanged upstream output permits early cutoff: after its repair, an observation-equal descendant can be unblocked without regeneration. For example, restoring an unchanged API signature can restore a patch whose entire observation is unchanged. A fresh episode on reactivation does not force a request if a compatible accepted cache record can be reused. A graph edge must cover guards, membership queries, structural expressions, validators, and selected context as well as textual prompts. A graph omitting one of these is unsuitable for either the locality or scheduling argument.

Proposition 6 (Observation-relative isolation) *If two states have equal job observations, contracts, attempt records, request activity, authorization, and local prerequisite usability, then they admit the same dispatch eligibility and the same set of locally valid proposal values for that job. This remains true when unselected source or context content differs.*

Proof Every local guard is a function of the listed data. Substitution of equals therefore preserves its truth and the validator's accepted-value set. □

This proposition concerns the controller and allowed inputs. It is not information-theoretic confidentiality: model pretraining, timing, logs, tools, or worker memory can reveal additional information unless independently controlled. Equal accepted-value sets do not imply equal sampled outputs or equal output probabilities.

6.2 Attempt and token bounds

Let E_j be the number of signature episodes of job j during a finite run interval, including episodes interrupted by an edit and episodes started by suspension/reactivation. A configuration change that changes η starts an episode. A deleted-and-recreated job starts another. Each episode admits at most k dispatched requests; persistent counters prevent an unchanged exhausted episode from receiving a fresh budget on the next controller pass. The failed stamp records diagnosis; with a persistent $n < k$ guard it is redundant for the numerical bound.

Theorem 7 (Generation bound) *For any finite execution interval starting with fresh episode budgets, summing over all jobs appearing in that interval,*

$$N_{\text{dispatch}} \le k \sum_j E_j. \tag{9}$$

If request u has at most p_u input tokens and a provider-enforced output cap ℓ_u, accounted generation tokens satisfy

$$T \le \sum_{u \ \text{dispatched}} (p_u + \ell_u). \tag{10}$$

Input caps include all instruction and metadata tokens. The output cap must cover all modeled output categories, including reasoning tokens if those are charged to T; a request to be brief is not an enforced cap. Each separately billed retry is counted as a dispatch. For common caps p, ℓ, this is at most $k(p+\ell)\sum_j E_j$.

Proof Within an episode, n starts at zero, dispatch strictly increments it, and no transition decreases it. The guard $n < k$ therefore allows at most k dispatches. Retries, timeouts, and obsolete responses do not reset n in that episode. Sum over episodes. The token inequality follows by summing the per-request caps, including requests later discarded. □

The bound charges actual observed episodes, not merely distinct signature values: an input sequence a, b, a, b may earn four budgets. Continuous editing can therefore cause unbounded work. A downstream job can also observe several upstream updates in one repair wave. The simpler bound $k|D|$ holds only for jobs processed once after their prerequisites stabilize, each encountering one new episode. The theorem bounds generation calls and tokens under caps, not matching, validation, global checking, network traffic, or total wall-clock time. A full footprint scan can cost $\Omega(|\mathcal{J}|)$ even when no generation is needed.

There is no unconditional speedup over a full-context baseline. Local observations may be large, many jobs may be affected, validators may be expensive, or a baseline may use fewer calls. The benefit proved here is suppression of generation for unchanged usable jobs and a dependency-episode bound. Empirical performance is a separate question.

7 Terminality and controlled reconfiguration

7.1 A sufficient scheduling discipline

Acyclicity must apply to all data dependencies, not merely arrows between agent names. Partition jobs and structural stages into a finite directed acyclic graph. A0 supplies the read order; A5 additionally requires finite instantiated work and stable stage processing. Jobs grouped in one stage are mutually independent; otherwise the stage is subdivided. Each node has one producer; its matches, guards, structure, observations, and validators read only external immutable inputs or earlier nodes. No output can influence its own or an earlier stage's existence, validation, or selection policy. The normalizer and validators terminate. A stage is stable after its predecessors are stable and it reaches either a usable result or an exhausted/blocked result.

Algorithm 1 is a conservative scheduler for one frozen configuration/input revision. It processes stages in topological order and keeps parallelism only among independent stages. It retains usable values from earlier revisions when observations agree. A dependent stage of a failed prerequisite is marked blocked without calling a worker. "Blocked" is a terminal controller classification for that revision, not satisfaction of the requirement.

Algorithm 1 Topological repair of a stable revision

Require: admissible C, frozen external revision X, budget $k \geq 1$

for stage s in a topological ordering **do**
  if some required predecessor of s is failed or blocked **then**
    classify s as blocked; continue
  normalize s, create/delete jobs, reconcile their signatures
  for job j in s with $\neg U_j$ **do**
    while $\neg U_j$ and $n_j < k$ and observation available **do**
      invoke guarded Dispatch(j); wait for response or controller timeout
      apply guarded resolution and update n_j, f_j
    if $\neg U_j$ **then**
      classify unavailable or exhausted j as failed/blocked
if Publishable(S) **then**
  publish the revision
else
  report local failure, blocked dependency, or global conflict

Theorem 8 (Finite terminal outcome) *Under A0–A6 and Algorithm 1, after external input and configuration stabilize the controller eventually reports either a publishable snapshot or explicit failure/blockage/global conflict. It dispatches at most $k|\mathcal{J}_*|$ requests during a new topological pass, where $\mathcal{J}_*$ is the set of all job instances appearing during that pass, including any transient instances. Successful publication follows if every required observation is available, every dirty job obtains a successfully committed proposal within its remaining budget, including successful typing, local validation, and complete candidate normalization, and the resulting snapshot satisfies G.*

Proof Order stages topologically. At the first stage, external inputs are stable; normalization produces finitely many jobs. A usable job needs no request. Each other available job either succeeds or consumes its finite remaining budget, since controller computations progress fairly and every dispatch eventually resolves or times out by A6. After the final external mutation a stable pass is resumed or restarted. Unavailable jobs are classified directly. Thus the stage becomes stable in finite controller progress. Inductively, the predecessors of each next stage are stable. If a prerequisite failed, the stage is classified blocked. Otherwise pure normalization and observation functions yield a fixed finite job set and each job has a fixed signature during its processing. The same finite-budget argument completes that stage. There are finitely many stages, so the final total predicate is evaluated. Independence within each stage prevents a sibling job from changing its observation. The request bound sums at most k per instantiated job; reused budgets can only reduce it. The additional hypotheses exclude all failure and blockage cases and establish the publication guard. □

Finite budgets establish terminality, not successful synthesis. Weak fairness of an arbitrary scheduler alone does not establish this theorem if contracts keep changing. Cyclic read dependencies are outside A0 and therefore outside the proved base semantics. In a hypothetical cyclic extension, a single Boolean value updated by $v \leftarrow \neg v$ on a feedback edge oscillates forever while each local proposal is valid. A global pass cap converts such an execution to explicit failure. More permissive convergence disciplines require an independent well-founded ranking or monotone finite-height domain proof; they cannot be obtained from freshness alone. Work on consistency-restoration networks motivates treating scheduling as a separate correctness concern [4, 6].

7.2 Safe configuration changes

An agent, transformation, permission, or validation-policy change is a configuration transformation $h : C \mapsto C'$. Its safety depends on what it does to the entire controller state. For example, adding a security reviewer creates new targets and obligations and extends the global completion predicate. It is incorrect to retain a previous claim that the earlier publication represents the current revision while the reviewer has no generated findings.

Theorem 9 (Transactional reconfiguration) *Let S satisfy the invariant of Theorem 2. A configuration transaction that checks A0–A4 for C', invalidates every changed observation or authority, reconciles structure and jobs, and recomputes completion produces a state satisfying that invariant under C'. If it additionally preserves A5 and eventually stops changing, Theorem 8 applies to the new revision.*

Proof The installed structural state is in the range of $N_{C'}$, hence satisfies (i). A retained usable record has the same effective observation and compatible authority; its validation evidence therefore still proves (ii)–(iii). Other records are historical or invalidated and cannot be consumed through the usability gate. Requests with changed job identity, contract, or authority are canceled, and unchanged requests must still pass the current commit guard, giving (iv). No historical publication is relabeled current, and any new publication must pass the recomputed predicate, giving (v). Atomic installation prevents readers from observing

mixed configurations. Terminality follows by applying the preceding theorem to the stable new configuration. □

This is a preservation theorem for checked transactions, not an algorithm deciding admissibility of arbitrary higher-order transformation programs. Pure finite rule templates make many checks decidable. General termination, arbitrary semantic validator soundness, and footprint adequacy remain proof obligations. Permission revocation cancels future commitment but cannot erase data already sent to a worker. Configuration rollback and external side-effect compensation require additional protocols.

7.3 Structured ingress, shared writers, and observation

Natural-language input can enter the controller through a separate lifting worker (a translator from text to a typed model) that proposes a typed source-view delta Δ. Treat this delta as untrusted input to Edit, rather than giving the worker direct structural write access. The ingress checker verifies source ownership, primitive and reference types, source invariants, and absence of writes into engine-owned target namespaces. It then constructs and normalizes the entire candidate transaction. The safety induction applies unchanged: rejected deltas leave state unchanged, and admitted deltas take the checked Edit case. This proves preservation of structural and coordination invariants, not fidelity of the delta to the user's intended meaning. That stronger claim requires an explicit input-to-model relation and a sound checker or human confirmation for it. Lifting calls have a separate budget and are excluded from Equation (9).

A shared-context update is also an authorized Edit. Give each context a monotonically increasing version and require the writer to submit its expected version. In the same atomic transaction, compare that version, install a new immutable snapshot, and increment the version. Two concurrent writers starting from version v cannot both commit against v: in either linearization order, the first successful update changes the version, falsifying the second comparison. A failed writer must reread and reconcile its proposal. This is an ordinary optimistic concurrency discipline [7]; it prevents lost updates under A3 but does not resolve semantic disagreement between writers. Dependency signatures still use selected content unless version numbers are observable inputs.

External observation can be modeled without making a transport authoritative. Extend state by W, containing both the external outbox and the remote observer projection. Committed controller events may be copied into W; export, loss, duplication, and remote delivery affect only W. Neither the controller's admission guards nor its authoritative acceptance evidence A may read W, and copy failure may not block a controller action.

Proposition 10 (Observational extension) *Erasing W maps every run of this nonblocking observation extension to a run of the controller, allowing stutters. Hence observer transport cannot falsify the controller's safety invariant under these extension rules.*

Proof An extended controller step has the same guard and authoritative effect as its base step, with only an additional optional copy into W. It projects to that base step. A pure observation/transport step changes only W and projects to a stutter. Initial projections agree; induction over steps proves the simulation and transfers safety. □

The proposition does not establish completeness or authenticity of a remote timeline. Digital signatures can authenticate a recorded sender without proving event completeness or correctness of its assertions. Context received from an external channel must re-enter through authorized, version-checked Edit; it cannot bypass the controller by being called an observation. Infinite observer activity must not starve controller execution, so progress still requires A6. This separates authoritative provenance, optional dissemination, and semantic validation.

8 Worked derivation and boundary cases

Initially request s has the business status accepted (distinct from a controller-accepted generated proposal), its endpoint has API signature `put(id,body)`, and its patch and checks are usable. Trace keys use the rule and stable request/clause identities. Clause c changes to require status 409. The endpoint's deterministic structure does not change. Its signature-generating job becomes dirty if its observation includes the clause; even if it returns the same signature, the patch observation changes because it also contains the clause. The clause-derived check is independently dirty. Objects for an unrelated request retain their observations and accepted pairs by Theorem 5.

Let the three dirty jobs be endpoint signature, patch body, and check oracle. Under a topological pass each sees one fixed new observation after its prerequisites stabilize. With $k = 2$, at most six requests are dispatched; successful first proposals use three. This is an arithmetic instance of Theorem 7, not a measured token result. If endpoint signature generation fails twice, the patch is blocked and the controller reports failure. It does not publish an old patch merely because its bytes are still available.

Suppose a shared security snapshot contains findings a for path traversal and b for authentication. First take fixed-item selectors: the patch reads a and the verification job reads both. Changing only b leaves the patch signature unchanged if the patch is not exposed to context version numbers. Alternatively, under a query selector, adding a new finding matching the patch's selector does change its observation, through set membership. Publishing a new context version with identical selected items changes audit pins but does not require regeneration. If exact version numbers are placed in the generation prompt, they must instead enter the signature, making such a change observable.

Now dispatch a patch request with observation o_1. An authorized context update creates o_2 before the response returns. Edit/reconcile cancels the old request. Its eventual answer cannot commit, even if it passes the old validator. A new request uses a distinct episode identifier. If the context later returns to o_1, a new episode is created; an answer to the original canceled request still cannot commit. This distinguishes exact content equality from request authority and explains why signatures alone do not provide idempotent delivery.

Table 4 Boundary cases and the corresponding obligation

Counterexample	Required discipline
Clause changes while signature stays equal	Adequate effective footprint
Answer returns after a source edit	Atomic current-signature and active-request checks
Two locally valid outputs disagree	Global consistency predicate before publication
Same failed request retried every pass	Per-episode attempts and failed-signature memory
New matching context item is added	Selection membership is a dependency
Validator or prompt changes, source does not	Contract identity in the signature
Feedback repeatedly flips a value	Acyclic schedule or independent convergence proof
Old request arrives after job recreation	Nonreused request and episode identities

Finally add a reviewer and a rule creating one assessment per endpoint. New match keys create fresh assessment jobs, and G now requires their success. Theorem 9 preserves safety, while current publication remains false until the new obligations are discharged. Removing the reviewer removes its owned jobs and traces, cancels pending requests, and updates G atomically. Any other view referencing its assessments must be repaired or the removal is rejected by structural admissibility.

Table 4 summarizes counterexamples essential to interpreting the claims. They identify specification or implementation obligations, rather than hiding failure behind an unconditional "correct by construction" label.

9 A fully discharged behavioral instance

Coordination invariants are useful even with weak validators, but a stronger domain claim requires a sound validator. We now discharge this obligation for a restricted artifact language. This gives a complete example in which generated behavior, as well as artifact bookkeeping, is verified.

Let a document have state $D = \{\mathsf{open}, \mathsf{archived}\}$ and events $E = \{\mathsf{read}, \mathsf{put}, \mathsf{archive}\}$. A generated proposal is exactly a total six-row decision table $t : D \times E \to \{200, 409\} \times D$. The output pair is (status, next state). The model covers archive state and response status; it does not represent payload contents, authentication, or persistent storage. A fixed trusted interpreter executes one table row per event and exposes no other side effects. The policy observation contains a bit h, where $h = 1$ requires rejection of put events in the archived state. For $h = 0$, such a put returns 200 and retains the archived state; only the response policy differs. Define the

reference function

$$f_h(d,e) = \begin{cases} (200, \mathsf{archived}) & e = \mathsf{archive}, \\ (409, \mathsf{archived}) & h = 1,\ d = \mathsf{archived},\ e = \mathsf{put}, \\ (200, d) & \text{otherwise.} \end{cases} \tag{11}$$

The validator checks the table's exact domain, primitive types, and $t(d,e) = f_h(d,e)$ for all six inputs. It rejects duplicate or missing rows. The generator is free to produce any syntax, but the decoder accepts only this finite table language. The footprint contains h, the event/state schema, the interpreter identity, and validator contract; the initial state is fixed to open. It excludes descriptive prose that has no role in this explicitly defined behavior.

Theorem 11 (Sound table generation and trace behavior) *For this artifact language, the footprint is adequate for extensional equality to f_h, and the exhaustive validator is sound and complete for that equality. If a published job has $h = 1$, every finite execution of its trusted interpreter from the specified initial state obeys: an archived document remains archived; every put while archived returns 409; and archive takes either state to archived.*

Proof The domain relation is $R(Z,t) \iff \forall (d,e) \in D \times E,\ t(d,e) = f_{h(Z)}(d,e)$ with the required total typed domain. Its truth depends only on the recorded policy bit and fixed schema/interpreter contract, giving adequacy. The validator enumerates exactly the six domain elements, so it accepts exactly this relation, proving soundness and completeness. By Theorem 2, current usable accepted tables satisfy that validator. For trace behavior, induct on the input word length. The empty word has the specified initial state. For the next event, equality to Equation (11) gives the same response and successor as f_1. If the predecessor state is archived, read and put retain it and archive also yields archived. The put case produces 409; the archive case always yields archived. These cases preserve the induction hypothesis for every next event. □

Figure 3 draws the resulting state machine. A change from $h = 0$ to $h = 1$ changes the effective signature even though the interface stays the same. Old pending tables cannot commit; an accepted current table has the stronger trace guarantee. This example obtains checked behavior by restricting the proposal language and trusting a small interpreter. It does not transfer that guarantee to unrestricted patches merely because they implement a similar interface.

10 Executable finite-instance validation

The mathematical proofs quantify over arbitrary finite views subject to their hypotheses. To exercise the asynchronous guards separately, we provide an explicit-state breadth-first explorer, `verification/explore.py`. It models one job, two source revisions, a episode numbers bounded by the environmental-change budget, two attempts per episode, active request identity, acceptance provenance, failure memory, deletion/recreation, contract revision, and nondeterministic valid/invalid resolution.

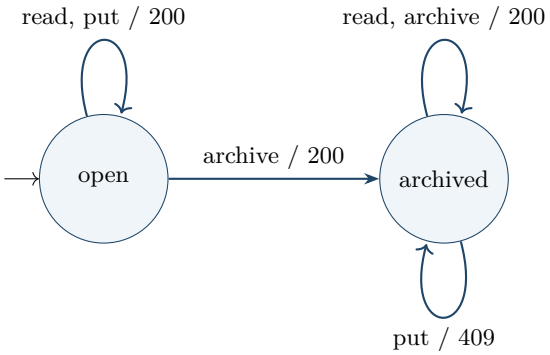


Fig. 3 The checked document policy for $h = 1$. Labels give input event and output status. The table interpreter preserves these transitions for every finite event sequence

The environment has a finite edit/reconfiguration budget. The explorer enumerates every successor defined in that abstraction, deduplicates states, and stores shortest counterexample traces.

The executable checks are intentionally narrow. Validator outcomes are abstract nondeterministic pass/fail choices, not executions of the decision-table validator. They test that no commit transition admits a value whose dispatch observation or contract differs from that transition's current state, that inactive responses cannot commit, that attempts never exceed the episode budget, and that a failed unchanged episode does not dispatch again. They also check nonvacuity by reaching accepted, exhausted, canceled-response, recreated-job, and changed-contract states. Deliberately mutated variants remove current-observation protection or failed-budget discipline. A counterexample in a mutant demonstrates sensitivity of the corresponding assertion; it is not a new proof of the unbounded semantics.

The explorer's results and exact state-space parameters are reported in its generated `results.json`. This first abstraction does not contain graph matching, multiple jobs, global constraints, arbitrary validators, continuous time, or storage failures. There is no proved abstraction map from a deployed runtime into this executable model. Consequently, finite-instance success supports debugging of the specified guards only. The refinement result of Theorem 4 is a mathematical result about the transition rules, not a claim that an existing implementation has been mechanically verified.

The baseline protocol exploration reaches 1,125 states including the initial state and examines 5,028 transitions for one job, two source values, two contract values, $k = 2$, and at most two environmental mutations. Coverage includes 230 accepted states, 168 exhausted states, 444 states with canceled requests awaiting delivery, and 35 recreated-job states. These are counts of states, not independent execution traces. Table 5 includes the mutation results. The single-defense variants demonstrate redundancy in this abstraction; they establish no progress guarantee.

The obsolete-response mutant's shortest trace is dispatch, edit, valid response. The budget-reset mutant's shortest trace is dispatch, rejection, dispatch, rejection, dispatch.

Table 5 Executed finite-instance exploration. Mutant counts stop at the first shortest counterexample

Variant	States	Transitions	Result
Protocol baseline	1,125	5,028	Exhaustive pass
Cancel omitted	569	2,352	Exhaustive pass
Commit recheck omitted	1,125	5,028	Exhaustive pass
Both defenses omitted	33	37	Counterexample
Exhausted budget reset	177	299	Counterexample
Interaction baseline	112	274	Exhaustive pass
Added review ignored	44	69	Counterexample

A second explorer, `verification/interactions.py`, studies two dependent atomic jobs and an added required reviewer, with Boolean output values and one external edit. This is a different atomic abstraction, so its size is not directly comparable to the protocol model. It reaches all 112 states and checks 274 transitions, including successful publication self-loops. It exercises blocked descendants, reuse after equal upstream repair, invalidation after changed repair, locally fresh global conflicts, and missing new-review obligations. Ignoring the added reviewer yields the four-step trace: add reviewer, accept upstream, accept downstream, publish. This experiment models the abstract accepted-update layer and cached records; it omits asynchronous delivery and structural normalization. Its exact report is `interactions-results.json`.

11 Related work and formal-method choices

Closest combination and concurrency foundation.

Stevens already connects software build, stamps, failures, and heterogeneous model consistency, proving soundness and incrementality for an extended build framework [8]. Our contribution is not that connection. We specify acceptance of asynchronous untrusted proposals under current contracts and authority, unsuccessful episode budgets, suspension, and configuration transactions. The atomic revalidation step is an application of optimistic concurrency control [7], rather than a new concurrency-control principle. The combined proof obligations identify what a generative coordinator must additionally preserve.

Shield synthesis supplies another close comparison: a synthesized shield corrects reactive outputs according to formal safety requirements [9]. Our controller instead defers or rejects primitive proposals, preserves provenance under change, and may report terminal failure. We establish neither instantaneous corrective synthesis nor minimum-deviation shielding. The finite decision-table instance demonstrates how stronger behavioral claims become available with a sufficiently restricted proposal language.

Execution enforcement.

Schneider characterizes policies enforceable by execution monitoring [1]. Runtime enforcement further studies how monitors modify executions to ensure specified properties; Pinisetty et al. give a timed-property theory with explicit correctness criteria [10]. Our controller also excludes invalid steps, but its state includes persistent generated artifacts, dependency episodes, and transactional configuration. We use untimed guarded transitions because the target claims concern freshness and admissibility, not deadlines. A timed-automaton extension would be appropriate for bounded response latency, which is not established here.

Model transformation and consistency.

ATL provides typed transformation rules and explicit execution support [5]. Stevens studies consistency across networks of heterogeneous models [4], and Klare and Gleitze analyze termination and expressiveness of network execution strategies [6]. These works make clear why pairwise consistency and global restoration cannot be conflated. Our setting uses directed, single-producer stages with nondeterministic primitive proposals. It does not establish the usual round-trip laws of bidirectional lenses (laws relating updates in opposite directions). The additional proof obligations concern rejected values, obsolete responses, and generation contracts.

Incremental computation and build systems.

Self-adjusting computation tracks dependencies and propagates changes [2]; build-system theory separates scheduling from rebuilding [3]. The observation signature is a rebuild key, and failed-attempt memory extends it with an explicit terminal unsuccessful outcome. Unlike deterministic rebuilding, two valid repairs may produce different primitive values. Our theorem therefore preserves admissibility and suppresses unnecessary calls relative to observations, rather than claiming equality with a fresh stochastic run. This distinction prevents importing deterministic incremental-correctness results without their hypotheses.

State-machine refinement and verification evidence.

IronFleet illustrates how state-machine refinement can connect protocol behavior to high-level specifications with mechanized implementation proofs [11]. We use a forward simulation at the coordination level and explicitly leave implementation refinement open. Recent work in this journal supplies further methodological context: Grosen et al. give symbolic timed monitoring algorithms [12]; Lu et al. use scenarios to guide hybrid-network verification [13]; Amilon et al. formalize correctness of program instrumentation [14]. Our use of untimed automata does not inherit their timed, hybrid, or instrumentation results.

Inductive invariants fit ownership and structural safety because each controller action has a local preservation obligation. Simulation fits asynchronous proposals because dispatch and delivery can be erased while accepted effects remain visible. Dependency equality fits repair locality because it states exactly when cached validation evidence is reusable. Topological induction fits terminality because earlier stages

stabilize before later stages execute. These techniques address different obligations; replacing all four with a state diagram would leave the main claims unsupported.

12 Limitations and research obligations

The trusted computing base, the components whose correct operation is assumed, contains normalization, signature construction, authorization, transaction isolation, validator execution, and publication. Bugs in any of these components can violate the assumptions. Recording provenance is not sufficient: the controller must enforce it at the write boundary. A concrete implementation should establish refinement to Section 4.2, including durable request identifiers, crash recovery, and transaction serializability. The present paper supplies no end-to-end machine-checked implementation proof.

The semantics treats generated code as inert data. Publication releases a checked snapshot; execution requires the separately stated interpreter or deployment assumptions. Sandboxed execution of validators is an assumption, not a consequence of primitive-only writes. A worker with independent filesystem, network, or deployment permissions could create effects outside the machine. Such effects require mediated tool actions and a larger transition system. Likewise, append-only audit records support reconstruction only if storage and identity are trustworthy; traces establish recorded derivation, not causal understanding or originality of generated content.

Footprint adequacy is the principal semantic boundary. Engineers must show that selected observations include all information needed by the desired relation, including negative dependencies and shared context. Parsers and finite test suites often establish only weaker relations. Theorem 11 discharges this obligation for one finite language. A compositional route to stronger claims is to require proof-carrying proposals checked by a trusted verifier and to express interface assumptions explicitly. Whether that is practical for a given artifact language requires further research and cannot be inferred from successful generation.

The acyclic discipline limits feedback and shared mutable targets. It defines the base usability semantics and supports the terminality proof, while excluding unrestricted collaborative editing and cyclic consistency restoration. Global conflict is reported after local repair; no theorem promises an automatic solution. Observations and input revisions can grow without bound, and the finite explorer does not establish scalability. Empirical performance claims or a broader implementation evaluation would need independent workloads, declared fault models, reproducible measurements, and baselines matched on context and validation strength.

13 Conclusion

A deterministic controller with ranked acyclic dependencies can coordinate untrusted generative proposals with explicit, conditional guarantees. Typed structural normalization protects object and reference integrity; exact effective signatures and atomic commitment exclude obsolete results; publication separates local acceptance from global consistency. Dependency episodes give an honest generation bound, and a finite acyclic discipline yields a terminal success-or-failure verdict after inputs stabilize. Transactional reconfiguration preserves these properties when its proof obligations are checked. The central remaining obligations are semantic footprint adequacy, validator soundness, and refinement of a concrete implementation to the controller machine.

Statements and Declarations

Research materials and data.

This is a theoretical study with an illustrative workflow and a finite-state validation artifact. No empirical performance or functional-correctness measurements are claimed. The accompanying explorers and their run reports are supplied locally with this manuscript; no public archive identifier has been assigned.

AI assistance.

Generative AI assisted development of the formalization, prose, executable exploration, and internal critical reviews. The reviews are simulated research assistance and do not constitute independent human peer review. The human author is responsible for checking the claims, proofs, references, and final submission.

Funding and competing interests.

Funding and competing-interest declarations must be confirmed by the author before submission; no declaration about undisclosed circumstances is made here.

References

[1] Schneider, F.B.: Enforceable security policies. ACM Transactions on Information and System Security **3**(1), 30–50 (2000) https://doi.org/10.1145/353323.353382

[2] Acar, U.A., Blume, M., Donham, J.: A consistent semantics of self-adjusting computation. In: Programming Languages and Systems (ESOP 2007). Lecture Notes in Computer Science, vol. 4421, pp. 458–474. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-71316-6_31

[3] Mokhov, A., Mitchell, N., Peyton Jones, S.: Build systems à la carte. Proceedings of the ACM on Programming Languages **2**(ICFP), 1–29 (2018) https://doi.org/10.1145/3236774 . Article 79

[4] Stevens, P.: Maintaining consistency in networks of models: bidirectional transformations in the large. Software and Systems Modeling **19**(1), 39–65 (2020) https://doi.org/10.1007/s10270-019-00736-x

[5] Jouault, F., Allilaire, F., Bézivin, J., Kurtev, I.: ATL: A model transformation tool. Science of Computer Programming **72**(1–2), 31–39 (2008) https://doi.org/10.1016/j.scico.2007.08.002

[6] Klare, H., Gleitze, J.: Termination and expressiveness of execution strategies for networks of bidirectional model transformations. Formal Aspects of Computing **35**(3), 1–35 (2023) https://doi.org/10.1145/3543845 . Article 15

[7] Kung, H.T., Robinson, J.T.: On optimistic methods for concurrency control. ACM Transactions on Database Systems **6**(2), 213–226 (1981) https://doi.org/10.1145/319566.319567

[8] Stevens, P.: Connecting software build with maintaining consistency between models: towards sound, optimal, and flexible building from megamodels. Software and Systems Modeling **19**(4), 935–958 (2020) https://doi.org/10.1007/s10270-020-00788-4

[9] Könighofer, B., Alshiekh, M., Bloem, R., Humphrey, L., Könighofer, R., Topcu, U., Wang, C.: Shield synthesis. Formal Methods in System Design **51**(2), 332–361 (2017) https://doi.org/10.1007/s10703-017-0276-9

[10] Pinisetty, S., Falcone, Y., Jéron, T., Marchand, H., Rollet, A., Nguena Timo, O.: Runtime enforcement of timed properties revisited. Formal Methods in System Design **45**(3), 381–422 (2014) https://doi.org/10.1007/s10703-014-0215-y

[11] Hawblitzel, C., Howell, J., Kapritsos, M., Lorch, J.R., Parno, B., Roberts, M.L., Setty, S., Zill, B.: IronFleet: Proving practical distributed systems correct. In: Proceedings of the 25th Symposium on Operating Systems Principles, pp. 1–17. ACM, New York, NY, USA (2015). https://doi.org/10.1145/2815400.2815428

[12] Grosen, T.M., Kauffman, S., Larsen, K.G., Zimmermann, M.: Efficient monitoring of timed properties. Formal Methods in System Design **69**(1) (2026) https://doi.org/10.1007/s10703-026-00498-5 . Article 2

[13] Lu, L., Shi, Y., Pan, M., Li, X.: Verifying hybrid automata networks guided by task scenarios. Formal Methods in System Design **68**(1) (2026) https://doi.org/10.1007/s10703-026-00492-x . Article 5

[14] Amilon, J., Esen, Z., Gurov, D., Lidström, C., Rümmer, P., Voorberg, M.: A program instrumentation framework for automatic verification. Formal Methods in System Design **68**(2) (2026) https://doi.org/10.1007/s10703-026-00493-w . Article 7